

Python Fundamentals — Interview Cheatsheet

Data structures · functions · classes · best practices · for the Just quant-engine interview

The winning move: answer fundamentals *through their engine* — "I'd model each instrument as a class implementing a common interface." See §5 for that one answer (it hits classes, ABCs, dataclasses, composition, type hints at once).

1. Data structures

Type	Mutable	Ordered	Use for	Key ops (Big-O)
list	✓	✓	sequences	index/append O(1) ; <code>in</code> /insert O(n)
tuple	✗	✓	fixed records, hashable (dict keys)	index O(1)
dict	✓	✓ (3.7+)	key→value, O(1) lookup	get/set/ <code>in</code> O(1) avg
set	✓	✗	uniqueness, fast membership	add/ <code>in</code> O(1) avg

- **collections**: `defaultdict` (auto-default), `Counter` (tallies), `deque` (**O(1)** both ends), `namedtuple`.
- **Comprehensions** (`[f(x) for x in xs if p(x)]`) — clearer than `map / filter`; **dict/set** comps too.
- **Generators** (`yield`, `(... for ...)`) — **lazy/streaming**, memory-efficient over big scenario sets.
- **Gotchas**: mutable **default arg** (`def f(x=[])` **✗** → use `None`); **shallow vs deep copy** (`copy` vs `deepcopy`); `list.__contains__` is **O(n)** (use a `set`); strings/tuples are immutable.

2. Functions

- **Signatures**: positional, `*args`, default, keyword-only (`def f(*, conf=0.995)`), `**kwargs`.
- **First-class**: pass/return functions; **closures**; **decorators** (`@property`, `@functools.lru_cache` to cache a base valuation, `@staticmethod`).
- **Pure functions** (no side effects, output depends only on inputs) → easy to test & reason about.
- **Type hints**: `def var(pnls: list[float], conf: float = 0.995) -> float:` → mypy-checkable, self-documenting.
- `lambda` for tiny inlines; prefer named funcs/comprehensions otherwise.
- **Generators vs lists**: generator = lazy, one pass, low memory; list = materialised, reusable, indexable.

3. Classes / OOP (the centrepiece)

- **Attributes**: instance (`self.x`) vs **class** (shared) — beware mutable class attributes.
- **Method kinds**: instance (`self`), `@classmethod` (`cls`, alternate constructors), `@staticmethod` (no `self / cls`), `@property` (computed/encapsulated, read-only by default).
- **Dunders**: `__init__`, `__repr__` (debug), `__eq__ / __hash__` (equality/keys), `__len__`, `__call__`, `__enter__ / __exit__` (context managers), `__iter__ / __next__`.
- `@dataclass` — auto `__init__ / __repr__ / __eq__`; `frozen=True` → immutable value object; `field(default_factory=list)` for mutable defaults.
- **Interfaces**: `abc.ABC` + `@abstractmethod` (enforced base class) or `typing.Protocol` (structural/duck typing — no inheritance needed).
- **Design**: **composition over inheritance**; `super()` for cooperative init; `__slots__` to cut memory/attr-lookup for many small objects.

3b. OOP in four pillars (say these if asked "what is OOP?")

Encapsulation · Inheritance · Polymorphism · Abstraction.

```
from abc import ABC, abstractmethod

class Instrument(ABC):
    # ABSTRACTION — a contract; can't instantiate
    @abstractmethod
    def revalue(self, mkt) -> float: ...

class Bond(Instrument):
    # INHERITANCE
    def __init__(self, n): self._n = n # ENCAPSULATION (_n = internal)
    def revalue(self, mkt): return self._n * mkt.df(1)

class Swap(Instrument):
    def revalue(self, mkt): return 0.0

# POLYMORPHISM — one loop, many types; never checks the concrete class
def total(insts, mkt): return sum(i.revalue(mkt) for i in insts)
```

- **Encapsulation** — bundle data+behaviour; hide internals (`_x` convention, `@property`).
- **Inheritance** — `Bond(Instrument)` reuses/extends; `super().__init__()` calls the parent.
- **Polymorphism** — call `revalue` on anything that has it (**duck typing**); the engine is open/closed.
- **Abstraction** — `ABC` + `@abstractmethod` (or `typing.Protocol`) defines the interface subclasses must meet.

Quick distinctions: `@classmethod` (gets `cls`, factories) vs `@staticmethod` (neither, utility) · `@property` (method used like an attribute) · **composition = has-a**, inheritance = is-a (prefer has-a) · **ABC** = explicit base subclasses inherit, **Protocol** = structural ("looks like it → it is").

4. Best practices (name these)

- **PEP 8** style + clear naming; **type hints** + `mypy`; **docstrings**.
- **Errors**: **EAFP** (`try/except`) over **LBYL**; **custom exceptions**; **context managers** (`with`) for resources.
- **Design**: **DRY**, **single-responsibility**, **immutability where possible**, avoid global/mutable state.
- **Testing**: `pytest`, pure functions, assert behaviour; small, composable units.
- **Ops**: logging over `print`; **vectorise numerics with numpy**; **don't prematurely optimise**; pin deps (`uv/venv`).

5. THE answer — design their engine in OOP

```
from abc import ABC, abstractmethod
from dataclasses import dataclass

@dataclass(frozen=True)          # immutable value object
class Market:
    rate_bump_bp: float = 0.0
    spread_bump_bp: float = 0.0
    def df(self, t: float, spread_bp: float) -> float: ...

class Instrument(ABC):           # the interface
    @abstractmethod
    def revalue(self, market: Market) -> float: ...

@dataclass(frozen=True)
class Bond(Instrument):         # add an instrument = a new class
    times: tuple[float, ...]
    cashflows: tuple[float, ...]
    spread_bp: float
    def revalue(self, market: Market) -> float:
        return sum(cf * market.df(t, self.spread_bp + market.spread_bump_bp)
                    for t, cf in zip(self.times, self.cashflows))

class Portfolio:                # composition, not inheritance
    def __init__(self, instruments: list[Instrument]): self._instruments = instruments
    def value(self, market: Market) -> float:
        return sum(i.revalue(market) for i in self._instruments)
```

Say: "Every instrument implements a common `revalue` interface (ABC/Protocol), so adding a Bond or CDS is a new class — the engine never changes (open/closed). Market and scenarios are frozen dataclasses (immutable, auto-generated boilerplate); the VaR loop just revalues the portfolio under each shocked market."

6. Likely questions → quick answers

- **list vs tuple** → mutable vs immutable; tuple hashable → dict keys / fixed records.
- **dict vs list** → keyed $O(1)$ lookup vs ordered sequence (`in` on list is $O(n)$).
- **@staticmethod vs @classmethod** → static: no implicit arg; classmethod: gets `cls` (alt constructors/factories).
- **@dataclass — why?** → auto `__init__`/`__repr__`/`__eq__`, frozen immutability, less boilerplate.
- **generator vs list** → lazy & memory-light vs materialised & reusable.
- **mutable default arg?** → shared across calls; use `None` then create inside.
- **how to add new instruments?** → the ABC/Protocol design in §5.
- **shallow vs deep copy?** → shallow copies references; deep recursively copies nested objects.
- **how do you test this?** → pure functions + `pytest` asserts on known values (your drills do exactly this).

Quick-ref

```
Big-O: list index/append O(1) | list `in`/insert O(n) | dict/set get/`in` O(1) avg | deque ends O(1)
Methods: instance(self) | @classmethod(cls) | @staticmethod() | @property (computed)
Immutable value object: @dataclass(frozen=True)
Interface: class X(ABC): @abstractmethod ... | typing.Protocol (duck-typed)
Mutable default fix: def f(x=None): x = [] if x is None else x
Context manager: with open(p) as f: ... ( __enter__ / __exit__ )
```

Practise the design hands-on in `python_oop_drill.py` (fill-in, graded): refactor the VaR engine into `Market / Scenario / Instrument / Bond / Portfolio / HistoricalVaREngine` with ABCs, frozen dataclasses and type hints. Key in `python_oop_drill_SOLUTIONS.py`.